Enabling AI-RAN Applications Through dApp Architecture Decisions: A Measured Study

Timothy O'Shea, Matthew Pennybacker, Andriy Kharchenko
DeepSig Inc., Arlington, VA, USA

***Abstract*—Distributed applications (dApps) bring AI to the microsecond-to-millisecond band beside the 5G distributed unit, but every public dApp framework realizes them the same way: an external process that receives an indication and returns a control message. That boundary is right for sensing and advisory workloads and cannot express a neural receiver that must finish inside a slot or a scheduling decision the MAC is waiting on. This paper treats the coupling between a dApp and the DU as a design axis with 2 legitimate ends, a tightly coupled C ABI inside the DU process and a protocol-standard E3AP association over SCTP that any process, container, or host can speak, and places the 3 dApp classes of the OCUDU platform on it. An audited corpus of 39 runtime AI-RAN use cases, with deadlines from 5G NR timing and byte counts at 51 and 273 PRB, shows that the 13 inline rows are inexpressible and the 9 bounded-control rows infeasible on the external boundary, while all 17 asynchronous rows are feasible at every position. The mechanisms each framework actually uses are then measured on 1 GPU host with a live cell under 1 pinned harness with raw percentiles: the asn1c APER codec and SCTP carrier, the OCUDU shared ring with eventfd try-lock and SEQPACKET supervisor, the validated direct-call intent path, and the ZeroMQ and device-to-host paths of the other frameworks. Those costs derive the released ABI: a stream for Class A, a validated call for Class B, and for Class C a per-use-case choice among a lease, a supervised ring, and a portable E3 client. The platform composes 4 dApps of all 3 classes on 1 over-the-air cell and records the checkpoints that let the argument be re-run.**

***Index Terms*—AI-RAN, dApps, E3, O-RAN, OCUDU, GPU, ABI, SCTP, ASN.1, shared memory, real-time interfaces**

I. Introduction

The O-RAN application hierarchy has 2 established tiers. rApps attach to the non-real-time RIC and act at management timescales of seconds and above; xApps attach to the near-real-time RIC through E2 and close control loops between 10 ms and 1 s [1], [2]. Neither can observe or act on the state of a distributed unit (DU) within a slot, a few slots, or a frame. Distributed applications (dApps) were proposed to fill that band [3], [4]: they run on the node that owns the data, consume observations that never leave it, and close loops below 10 ms.

Every published dApp framework realizes that idea the same way. The dApp is a separate process; the DU exports an indication, the process computes, and a control message comes back [4]–[6]. This is an *observer with a reply channel*, and it is the right shape for spectrum sensing, interference detection, or integrated sensing and communication (ISAC), where a result 2 ms late is still useful and a dropped one is harmless. It is the wrong shape for 2 other kinds of work the AI-RAN literature also files under "dApp": a neural channel estimator or receiver that must finish inside a PUSCH slot over tensors a GPU already holds, and a scheduling or link-adaptation decision the MAC is waiting on now. For the first, the export is the cost; for the second, the process boundary is the risk.

The companion papers define 3 dApp classes from a use-case corpus [7] and describe the embedded runtime that implements them inside the OCUDU DU [8]. This paper answers the question they leave open: *why these interface mechanisms, at these costs, and why offer more than one*. It measures the mechanisms the 3 public frameworks actually use, at the payload shapes the use cases produce, on the platform where the released runtime runs, and contributes:

1) a **coupling spectrum** for dApp interfaces with 2 legitimate ends, a tightly coupled C ABI inside the DU process and a protocol-standard out-of-process carrier (E3AP over SCTP), on which the 3 classes are positions and Class C spans 3;
2) an **audited corpus** of 39 runtime AI-RAN use cases with deadlines from 5G NR timing, byte counts at the live and envelope shapes, and a verdict on what an observer-only framework, a GPU-export framework, and an in-process ABI can each express;
3) **measurements of the real mechanisms** under 1 pinned harness with raw percentiles, on a quiet host and with the DU on the air: the asn1c APER E3AP codec and SCTP carrier, the OCUDU shared ring, SEQPACKET supervisor, and validated direct call, jBPF, and the ZeroMQ and device-to-host paths of the other frameworks;
4) a **derivation of the released ABI** from those measurements, including the case for keeping the protocol-standard end as a first-class Class C option; and
5) **validation**: 4 dApps of 3 classes composed on 1 over-the-air cell, with the runtime's per-class checkpoints.

Every number in the paper is produced by the accompanying suite or by the released runtime, and every mechanism named is the one the runtime ships.

II. Prior Frameworks and the Boundary They Assume

Three bodies of work define what a dApp is today; each is examined for the boundary it places between the DU and the

*Preview version. This paper accompanies a preview release of the platform and its benchmark suite and will receive updates as the working group's review, new use cases, and measurements land.

application, because that boundary decides which use cases it can express at all.

Northeastern dApp framework. Lacava et al. [4] pair an E3 agent embedded in OpenAirInterface with dApps that run as separate processes. Management uses E3AP, an ASN.1 service model compiled with asn1c in the E2 tradition, carried over ZeroMQ or TCP by default; the 2026 revision publishes the codec as a vendor-neutral library with SCTP among its link layers [9]. Data moves on a second plane: the agent publishes indications through ZeroMQ or a shared-memory reference, Python dApps subscribe, usually keeping only the newest message, and control actions return the same way to be applied at a safe point. The design is explicitly nonblocking; spectrum sharing and interference detection run on it [6], [10], and jBPF has been evaluated as a verified in-process hook for small functions [11].

NVIDIA Aerial dApps. Villa et al. [5] add a dApp path to the CUDA-accelerated Aerial L1: a GPU-side export copies a slot's IQ or channel-estimate tensor out of the PHY's buffers to a second context, where an inference model runs and reports through a host process, within a budget of a few milliseconds. Its defining property is an export that leaves the PHY's own stream untouched.

Vendor accelerators. Commercial L1 stacks expose configurable pipelines and, increasingly, neural components [12], [13], but in-pipeline replacement of a stage by third-party code is not a supported contract in any of them; the standard-compliant neural receiver of [14] ships as a vendor feature.

The first 2 groups share 2 properties: the dApp lives outside the L1 process, and the L1 owns the clock, so the application sees a copy and replies when it can. These make the frameworks portable, restartable, and safe to write in Python, and they fix the set of use cases the frameworks can express: a neural equalizer cannot sit on the far side of an export, and a scheduler decision cannot be applied "at a safe point later" when the slot is now. OCUDU keeps the observer boundary, implements E3AP over SCTP with the same asn1c toolchain, and adds 2 positions on the coupling spectrum so that the rows the boundary cannot express have a home.

III. Two Axes, Three Classes, One Spectrum

A. The Axes That Change the Interface

For every use case in the corpus of Section IV, 2 properties decide the interface before any model is chosen. The *blocking contract* states who waits: in an *inline* contract the PHY cannot progress until the function returns; in a *bounded* contract the MAC or PHY control loop waits but has a conventional decision it can take at a deadline; in a *nonblocking* contract nobody waits and the result is consumed later or dropped. The *payload residency* states where the bytes are: grids, channel estimates, and LLRs are produced in GPU memory by the accelerated PHY and stay there until the transport block is decoded; scheduler inputs are compact host structures the MAC already holds; spectrum windows and SRS responses are large and GPU-resident but read a few times per second.

The 3 classes of [7] are the 3 non-empty combinations. Class A, *Resident Inline L1*, is inline over resident tensors; Class B, *Bounded Real-Time Control*, is bounded over compact host state; Class C, *Asynchronous Observation and Advisory*, is nonblocking over either. The missing combinations are the ones physics excludes: an inline contract over exported data pays a copy and a wake on the slot's critical path, and a nonblocking contract over resident tensors is Class C with a lease.

B. The Coupling Spectrum

Fig. 1 presents the classes not as 3 boxes but as positions on 1 axis, the degree of coupling between the dApp and the DU, because that axis explains both the mechanism choices and the reason the platform keeps more than one.

At the **ABI-standard end** the dApp is a shared object loaded into the DU process and bound to a frozen, size-tagged C interface; the DU passes device pointers, a CUDA stream, and a deadline, and the dApp writes into DU-owned buffers and returns, with nothing copied or serialized. The cost is trust: the module runs with the DU's privileges and the ABI must not change under it. Class A, Class B, and the in-process Class C path live here. At the **protocol-standard end** the dApp is a process, a container, or a remote host that speaks a wire protocol. E3AP over SCTP is that protocol: the carrier the O-RAN E2 interface standardized [15], [16], one the Northeastern codec library supports, and the one several large vendors prefer for RAN control planes because their tooling, security posture, and conformance practice already exist for it. The cost is a codec and a kernel round trip per message; what it buys is vendor neutrality, language independence, crash isolation, and placement wherever the operator's security model requires. Between the ends sits the **supervised** position: a separate process on the same host sharing memory with the DU, paying 1 copy and 1 wakeup for crash isolation and restart.

Three consequences follow. The classes are not a ranking: a position is right or wrong for a use case. Class C is not "observer only": its contract permits every position, and it closes loops through a validated latest-context cache and through E3 control actions. And *optionality* is itself a requirement: a group already running Python dApps over E3 should be able to point them at OCUDU unchanged, a vendor shipping a neural receiver needs the left end, and an operator with a hardened-container policy needs the right end for anything it did not write. Section V prices each position so the choice can be made per use case.

IV. The Use-Case Corpus, Audited

The corpus of [7] listed 45 use cases drawn from the O-RAN nGRG report on dApps [17], the dApp literature [4]–[6], [10], and AI-native PHY work. Every entry was re-examined against the 5G NR timing that bounds it (a 500 μs slot at 30 kHz, retransmission DCI after $k_2 \geq N_2$, SRS periods of 5–40 ms, beam-failure timers of 10–200 ms), the bytes it moves at the released testbed shape (51 PRB, 2 receive ports) and at the 273-PRB, 4-port envelope, and the contract the released runtime offers. Rows mixing a per-slot half with a policy half

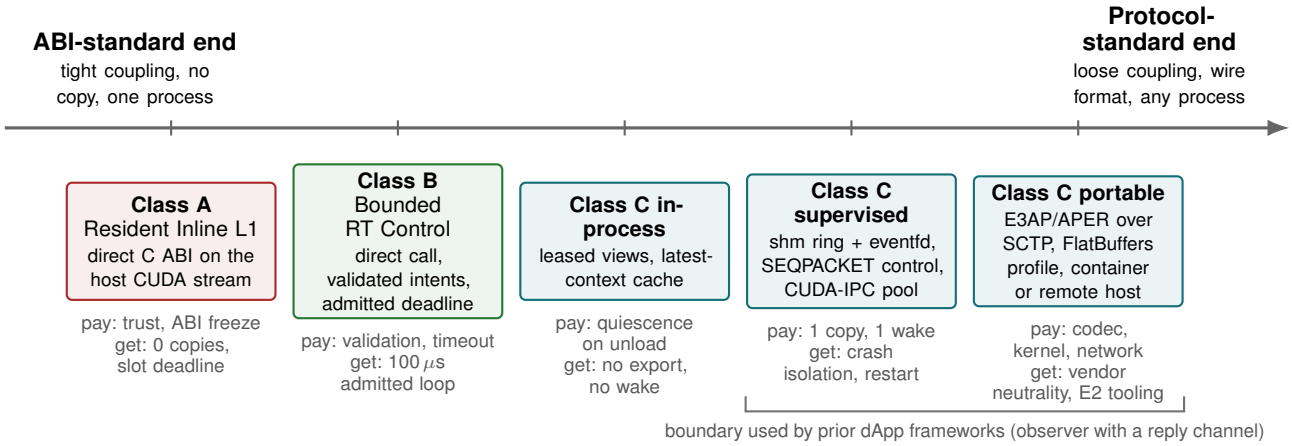


Fig. 1. The coupling spectrum. The 3 classes are positions on 1 axis, and Class C spans 3 of them. Prior frameworks occupy the right-hand half; the left-hand positions are what make inline and current-slot use cases expressible. Each position states what the dApp author pays and what the placement buys.

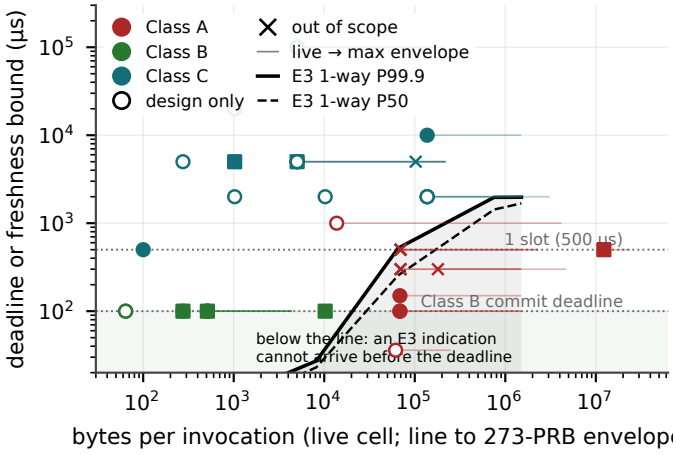


Fig. 2. The 39 runtime rows by bytes per invocation and deadline. Filled markers are shipped or live, hollow are design only, crosses are out of scope. The black lines are the measured one-way delivery of an E3AP indication (APER over SCTP), P99.9 solid and P50 dashed: a row below the solid line cannot be served across that boundary within its deadline.

were split, rows whose input the DU never sees or whose actuator sits inside the RU were moved out, and 10 rows the platform made possible, such as the observation-to-avoidance composition, were added. The audited corpus has 39 runtime rows (13 Class A, 9 Class B, 17 Class C), 9 management rows acting through typed configuration or RAN control, 1 research row that needs a multi-DU substrate, and 5 compositions that flow only through host-owned caches and authorities. Of the runtime rows, 14 are shipped or demonstrated live, 12 are design only, and 13 are out of scope for the released platform; the full tables are in the study repository, and Fig. 2 places every row against the 2 axes and the measured cost of the observer boundary.

Three facts carry the argument, and the verdicts behind them use the byte and deadline arithmetic of the corpus tables and the measurements of Section V. No Class A row is expressible in either external framework, for a reason unrelated to speed: an indication carries data outward, and nothing brings a channel estimate, an equalized tensor, or an LLR tensor back into the PUSCH chain of the same slot. Every Class B row has a return path in the observer-only framework, the E3 control action, whose cost Section V-D shows to be a multiple of the budget even on loopback with the real codec, and none in the export framework, which documents no decision-boundary hook. Every Class C row is feasible in all 3 frameworks, which is the case for keeping the protocol-standard end: for 17 of 39 rows an out-of-process E3 client is a correct implementation, and removing it would trade portability for nothing.

Table I lists the rows that most sharply separate the frameworks, with the mechanism each would have to use and its measured or published cost. Its Class C rows show where the platform's contribution lies once expressibility is settled: producer-side cost, survival across consumer restarts, and the 2 return paths by which an observation re-enters control (the interference map consumed by the scheduler and the quiet-period reservation admitted by a host authority) without a message on the decision path.

V. Measured Interface Costs

A. Method

All rows were measured on 1 NVIDIA DGX Spark (GB10, 20-core Arm host, coherent CPU–GPU memory) under 2 conditions: *quiet*, with the host otherwise idle, and *contention*, with the released OCUDU cell on the air on the same host and sharing its GPU. The text and figures report the quiet run; Table II sets the 2 side by side for the rows the argument rests on. Benchmarks ran pinned to the 6 cores the DU does not use, with memory locked, a 10 % warmup, and raw nearest-rank percentiles with no tail trimming; `CLOCK_MONOTONIC` above 1 μs and the cycle counter below it. Every CSV row carries its contract, placement, copies and system calls on the measured path, and an evidence kind, *measured* or *published baseline*. Where earlier drafts of this study modeled a cost, for example E3 as a fixed header plus a per-byte slope, the real mechanism

TABLE I

DISCRIMINATING ROWS OF THE AUDITED CORPUS. COSTS ARE P50 UNLESS STATED; *meas.* ROWS COME FROM THE SUITE IN SECTION V OR FROM THE RELEASED RUNTIME, *publ.* FROM THE CITED PAPERS. "INEXPRESSIBLE" MEANS NO RETURN PATH INTO THE CONSUMING STAGE EXISTS.

Row	Bytes, budget	Observer-only external (E3AP/SCTP + msg. data plane)	External GPU pipeline (D2H or D2D export)	OCUDU resident or bounded (in-process ABI)
A-03 neural receiver to LLRs	1.47 MB in, up to 4.4 MB out per slot; $\leq$500 μs occupancy	Inexpressible. For scale, a 734 KB indication costs 1.43 ms one way with the real APER codec over SCTP (meas.)	Inexpressible. Export of 1.47 MB to host costs 73.5 μs before inference (meas.)	Feasible at the live shape: 81.6 μs / 112 μs P50 / P99.9 (meas.), 92.5 / 105.1 μs runtime checkpoint; 273-PRB kernels not yet qualified
A-02 neural equalizer	68.5 KB in (live); same-invocation consumption	Inexpressible	Inexpressible	Feasible: 45–52 μs P50; 260,000+ invocations on air, 0 fallbacks
B-01 scheduler intents	$\leq$4.2 KB in, $\leq$2.3 KB out; 100 μs to commit	Infeasible under DU load: SCTP loopback round trip 15.6 μs P99.9 quiet, 181 μs with the DU on the air (meas.); about 400 μs in published loops	Inexpressible: no decision-boundary hook	Feasible: direct call 0.288 μs P99.9 quiet, 5.22 μs with the DU on the air, 20,000 validated calls (meas.); 3.6 μs on the runtime
B-04 avoidance from the C-01 map	$\leq$275 B context; 100 μs to commit	Infeasible: the map is an indication, its consumption is the B-01 path	Inexpressible	Feasible: latest-context cache, 3 bounded acquire attempts; demonstrated live
C-01 spectrum sensing	137 KB (live) to 1.47 MB per slot; no deadline	Feasible: 1.5 KB decimated indications in published loops; a full grid costs 1.68 ms one way (meas.)	Feasible: about 150 μs C API, 350 μs gRPC at 717 KB per slot (publ.)	Feasible on all 3 positions; producer critical section 2.34 μs P99.9, independent of the subscriber (meas.)
C-02 quiet reservation	about 100 B intent; $\geq$1 slot ahead	Feasible as an E3 control message; admission and duty bound absent from the published procedure set	Inexpressible: no scheduler authority	Feasible: host reservation authority with clamp and duty budget; shipped
C-03 SRS-ISAC	$\leq$210 KB per SRS occasion; $\leq$ SRS period	Feasible: 255 μs one way at 64 KB (meas.)	Feasible: about 2.3 ms cuSense (publ.)	Feasible: CUDA-IPC pool, 1 device-to-device snapshot; demonstrated live

is now measured, and rows once labeled inter-process that ran inside 1 process now fork.

The mechanisms are the ones the frameworks ship. The E3 rows link the OCUDU asn1c APER codec and SCTP transport from the SDK, with the message families and 64 KiB fragmentation of the E3AP wire contract, and the Northeastern libe3 codec beside it. The OCUDU inter-process rows use the released ring layout (128 B header, 64 B slot headers, 64 B-aligned stride, try-lock guard, eventfd signal) and its SEQPACKET control channel with credential checks on every message. The Northeastern-style rows use ZeroMQ publish/subscribe over `ipc://` across a fork with the high-water mark and conflating-subscriber options that framework's Python dApps use; the NVIDIA-style rows use a device-to-host copy into pinned memory, a shared-memory hand-off, and a ZeroMQ notification. Payloads are the corpus's: 34 and 68 KB for a 51-PRB slot as 16-bit and 32-bit complex, 734 KB and 1.47 MB for the 273-PRB, 4-port envelope, 64 KB for an SRS occasion, and a 16-candidate scheduler request of 3.2 KB returning 1.2 KB of intents.

B. The Protocol-Standard End: APER and SCTP

Fig. 3 gives the cost of the protocol-standard end with nothing modeled. The codec rows encode and decode real E3AP PDUs with the OCUDU asn1c APER codec: setup, subscription, indications with bodies from 64 B to a fragmented 734 KB grid, a 2.2 KB scheduler control action, and an acknowledgment. Two regimes appear. For control-sized messages the cost is the envelope, 2.27 μs to encode and 2.29 μs to decode a control action, independent of the few hundred bytes of body, and the Northeastern codec tracks it closely. For indications the cost grows linearly at 0.87 ns per byte because APER copies an `OCTET STRING` body bit-aligned: a 64 KB SRS indication costs 57.8 μs and a 734 KB grid 681 μs to encode, with decode near 0.97$\times$ the encode cost. The FlatBuffers data profile on the second association costs 42.1 μs to encode and 11.3 μs to decode for the same grid, which is why the platform has 2 associations.

TABLE II

THE SAME ROWS UNDER THE 2 HOST CONDITIONS (μS). QUIET: HOST OTHERWISE IDLE. CONTENTION: THE RELEASED CELL ON THE AIR ON THE SAME HOST, SHARING THE GPU, WITH THE BENCHMARK PINNED TO THE CORES THE DU DOES NOT USE.

	Quiet		Contention	
Row	P50	P99.9	P50	P99.9
A GPU-resident inline, 1.47 MB grid (floor)	58	119	13	801
B direct ABI, validated call	0.27	0.29	0.27	5.22
B same request over SCTP loopback	13	16	13	181
C ring producer critical section, 68 KB	1.97	2.34	1.97	167
C ring consumer wake, 68 KB	3.92	99	4.91	1.62 ms
C SEQPACKET control RTT/2, 256 B	3.20	4.26	3.25	69
C E3AP indication 64 KB one way	255	520	207	1.57 ms
C E3AP control round trip	11	13	12	492

The carrier rows send those PDUs between 2 pinned processes over an SCTP one-to-one association on loopback, 1 PDU per message, fragmented at 64 KiB as the E3AP wire contract requires, and time one-way delivery through reassembly and decode. A 1.5 KB indication arrives in 10.5 μs at the median; the 734 KB grid, 12 fragments, in 1.43 ms (1.97 ms at P99.9); the 1.47 MB 4-port grid in 1.68 ms. A control round trip on stream 0 with the real codec at both ends is 10.9 μs at the median and 12.6 μs at P99.9; the published loops of about 400 μs [4] sit between these compiled figures and the same path through a Python decoder.

These numbers draw the line on the corpus map of Fig. 2. Every Class C row lies above it, most by an order of magnitude, so an E3 client is a correct implementation of all 17; every Class A and Class B row lies below it. That is the whole

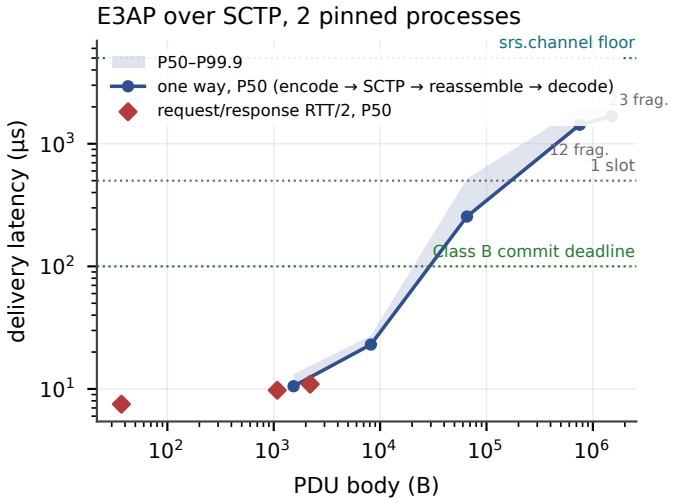


Fig. 3. The protocol-standard end, measured with the real codecs and carrier. One-way delivery of E3AP indications (asn1c APER encode, SCTP send, reassembly, decode) over an association between 2 pinned processes, and the control-plane request/response exchanges, against the deadlines that matter for each class. Codec costs alone are given in the text.

case for the right-hand end: it costs nothing the asynchronous rows cannot afford, and no codec optimization moves the line, because the line is set by the copy, the wake, and the association.

C. Class C Egress: The Producer Rule

A Class C publisher runs on the PHY thread. Whatever the consumer does, the producer's cost per slot must stay bounded and small, and the data the consumer eventually sees must have a known age. Fig. 4 tests both properties for the OCUDU positions and for the ZeroMQ publish/subscribe path of the observer frameworks, at the live 68 KB slot.

The supervised ring is the released layout in a 2-process harness: try-lock guard, reclaim read, memcpy into the next 64 B-aligned slot, release store, eventfd write, 3.0 system calls per publication. Its critical section costs 1.97 μs at the median for 68 KB and 23.9 μs for the 1.47 MB envelope, which is the memcpy; when the ring is full the producer returns in 0.144 μs with a counted drop and never waits. In the slot-timed run the producer publishes every 500 μs for 20,000 slots while the consumer either drains promptly or sleeps 5 ms on every 8th slot. With the sleeping consumer the ring drops 4457 slots across 1942 stalls, about 2 per stall, which is exactly the excess of a 5 ms stall over an 8-slot ring, and the age of the newest data the consumer sees is bounded at 3.15 ms median and 5.41 ms P99.9 by the ring depth. An unsubscribed stream costs the producer 1 relaxed atomic load, 0.032 μs P50, 0.048 μs P99.9.

ZeroMQ over `ipc://` publishes the same 68 KB in 1.82 μs at the median, the same producer cost; the difference is what happens to the data. With the default high-water mark the socket queues everything a slow subscriber has not read, so the age of the data it eventually sees grows to 981.95 ms at the median and 1045.61 ms at P99.9, with no drop reported; with the conflating subscriber option the Python dApps use, the age stays at 1 slot but 10588 slots are discarded inside the socket with no count anywhere. The ring is not faster than the socket; it is *accountable*: staleness is bounded by a configured depth and every drop is counted, which is what an operator needs to know whether an advisory loop is still advising. On the quiet host the ring producer's P99.9 is 2.34 μs; with the DU on the air it is 167 μs, and the consumer's wake tail grows from 99.4 μs to 1.62 ms (Table II), because the bench cores then also carry DU worker threads and enter deep idle states between slots.

The SEQPACKET control channel, credentials verified on every receive, completes a 256 B request/response in 3.20 μs at the median and 4.26 μs at P99.9 and a heartbeat in 3.06 μs, against 8.52 μs for ZeroMQ request/reply: the cost of noticing a dead worker within 1 heartbeat and restarting it without touching the ring. The consumer's wake from eventfd to a leased slot is 3.92 μs at the median and 99.4 μs at P99.9. The native lease costs the producer only the unsubscribed check.

D. Class B: One Request, Six Boundaries

The Class B question is whether a process boundary can sit inside a 100 μs decision. The same request, a 16-candidate scheduler input of 3,240 B (the released `scheduler_input_v1` with 2 275-bit masks) returning 16 intents of 72 B, is carried over 6 paths that all invoke the same dApp function through the released interface's `invoke` pointer: a direct call with the DU's admission checks (structure size and ABI major, candidate identity, allow flags, PRB range, MCS bound, deadline); a heap SPSC pair between 2 pinned threads; a shared-memory SPSC across a fork; ZeroMQ request/reply over `ipc://` and TCP loopback; and an SCTP one-to-one association on loopback with real message boundaries, the carrier an E3 control action would use. Each path runs 20,000 requests on separate pinned cores under 3 conditions: the quiet host, the quiet host with a 50 μs-on, 50 μs-off busy thread on the responder's core, and the host with the DU on the air.

Fig. 5 shows the result. On the quiet host every carrier meets the deadline: the direct call completes in 0.272 μs at the median and 0.288 μs at P99.9 with validation included, the shared-memory SPSC in 0.784 μs and 0.944 μs, SCTP in 13.1 μs and 15.6 μs, ZeroMQ over `ipc://` in 18.0 μs and 21.3 μs. The antagonist adds about 50 μs to every tail (65.8 μs for SCTP, 73.4 μs for ZeroMQ, 53.2 μs for the direct call, whose caller shared the contested core); all still fit. With the DU on the air the picture changes: the direct call moves to 5.22 μs at P99.9 and the SPSC to 16.8 μs, while the message paths move to 181 μs (SCTP) and 787 μs (ZeroMQ) and miss the deadline on 0.35 % and 0.69 % of requests with the responder core otherwise idle. APER encode and decode at both ends of the SCTP path add up to 22.2 μs at the median on the quiet host, a floor for a compiled E3 control loop; the published loops of about 400 μs [4] include a Python decoder and the agent's data plane, and report no tails.

The reading is not that ZeroMQ or SCTP are slow; on an idle host they would do. It is that a bounded contract is defined

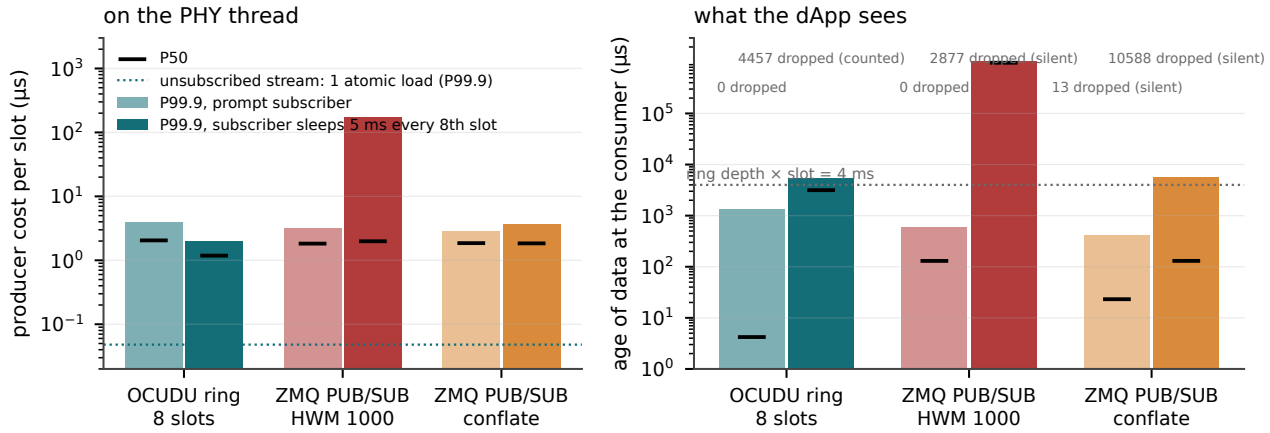


Fig. 4. Producer-side cost per 68 KB slot publication, P50 (tick) and P99.9 (bar), with a prompt subscriber and with a subscriber that sleeps 5 ms every 8th slot. The OCUDU ring's producer cost does not move; a full ring is a counted drop. ZeroMQ's producer inherits the subscriber's backlog.

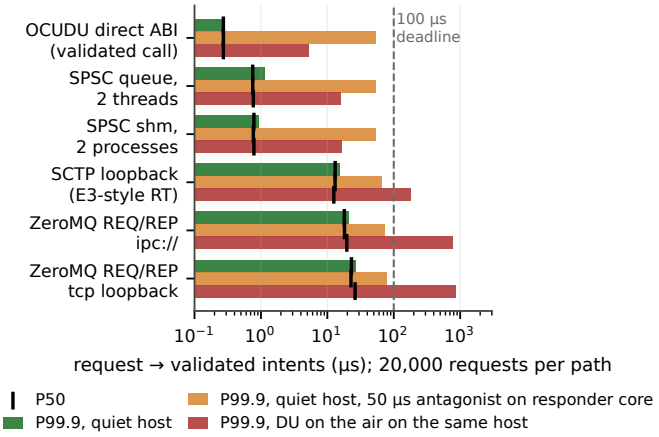


Fig. 5. One Class B request (3.2 KB in, 1.2 KB of intents out) over 6 boundaries, 20,000 requests each, under 3 host conditions. Bars are P99.9, ticks are P50; the dashed line is the admitted deadline. On the quiet host every path fits; with the DU on the air only the paths without a wake keep their tails.

by its tail under the load the DU itself creates, and the tail of any path with a wake in it is then set by the kernel and by whatever else the DU runs. The direct call is the one path whose tail stays inside the budget on the host that matters; the price, validating 72 B intents, is small (Section VI).

E. jBPF: A Verified Hook, and Why It Was Not Adopted

jBPF [11] runs verifier-checked codelets inside the DU at named hooks, the one prior mechanism that is not an observer with a reply channel, and the suite measures it as shipped. The hook is cheap, 1.54 μs for a 64 B context and 2.61 μs for 16 KB, rising to 95.4 μs for a 1.47 MB grid the codelet must read in full. The limit is what the codelet can do with its result: its output goes into jBPF's own ring, which an I/O thread drains for a consumer, so a request/response through the hook costs 152 μs at the median, set by that thread's polling interval, and in every published use the consumer is an external process, so the tap adds a copy into the ring before the copy across the boundary. The verifier also excludes device pointers and CUDA calls, which Class A needs, and bounds the loops a scheduler policy would write. The platform therefore keeps jBPF as an evaluated option for tiny Class B taps and adopts, for the same contract, a direct call whose output lands in the caller's structure and is validated field by field, with no ring, no I/O thread, and no second copy.

F. Class A: What an Export Costs Before the Model Runs

The Class A rows are inexpressible on an external boundary for the structural reason of Section IV, so the measurement answers a narrower question: were a framework extended with a return path, what would the export alone cost before any inference ran? On the coherent-memory host a device-to-host copy of a 1 MB grid into pinned memory takes 73.5 μs at the median and 80.6 μs at P99; the full NVIDIA-style path (copy, shared-memory stage, ZeroMQ notification) takes 224 μs at the median and 306 μs at P99 for the 1.47 MB 4-port grid, against a 300 μs planning budget and before the return copy that no framework provides. The in-process figure is the cost of enqueueing the dApp's kernels on the lane's stream and recording the completion event, measured with the released packages on the exclusive GPU: the channel estimator submits in 19.9 μs at the median at the live shape, and the reference receiver at the live shape (51 PRB, 2 layers, 256-QAM) completes in 81.6 μs / 112 μs P50 / P99.9, inside its 150 μs qualification deadline. At the 273-PRB envelope the same receiver submits in 45.9 μs but completes in 272 μs / 523 μs, so the reference kernels, not the interface, are what remains to be qualified at that shape. The framework-path suite shows the same thing at every payload from 256 KB to 23 MB: residency costs a launch, export costs a copy proportional to the tensor, and at the 64-port envelope the copy alone (about 17.4 ms at the measured 11 Gb/s device-to-host rate at 23.5 MB) exceeds the slot.

VI. Design Consequences: The Released ABI

Each mechanism in the released runtime [8] can now be stated as a consequence of a measurement in Section V and a row class in Section IV. Table III summarizes; the text gives the reasoning.

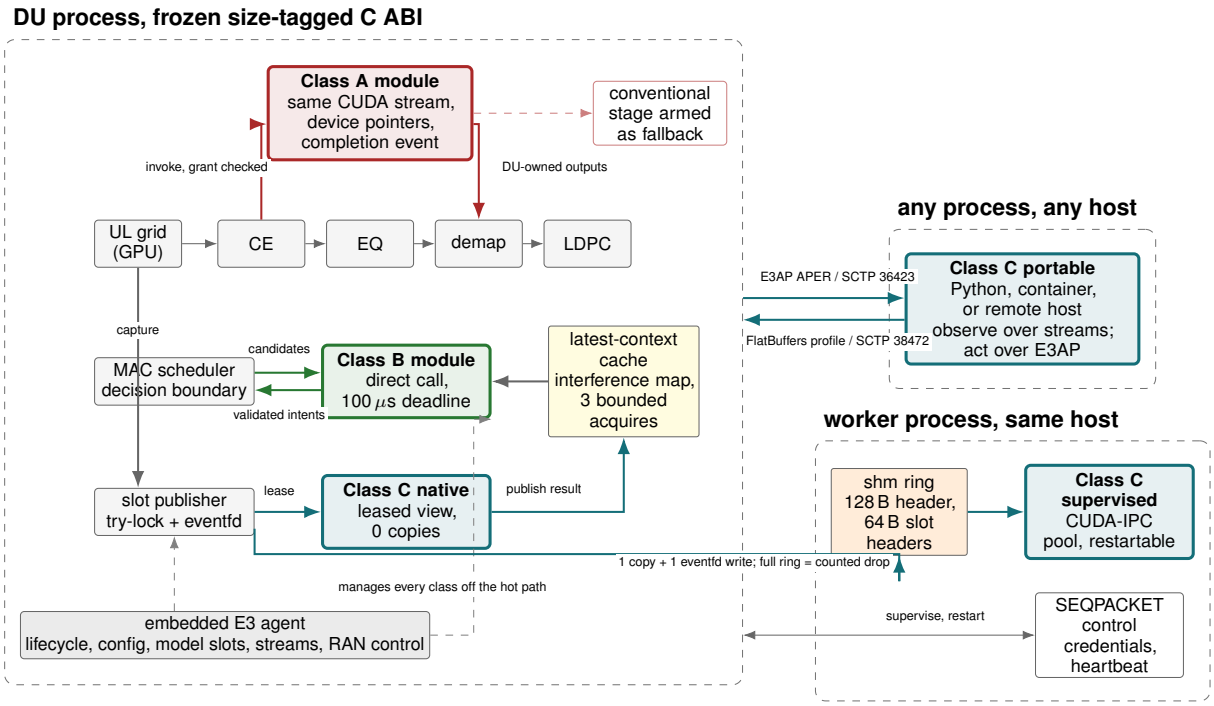


Fig. 6. The 3 classes inside and around the OCUDU DU. Class A is invoked on the PUSCH lane's own stream and writes DU-owned outputs; Class B is a direct call at the scheduler's decision boundary returning validated intents; Class C has 3 positions: a leased in-process view, a supervised worker fed by a shared ring, and a portable E3 client over SCTP. Observations re-enter control only through host-owned caches and authorities, and 1 embedded E3 agent manages every class off the hot path.

TABLE III
MECHANISM ADOPTED PER CLASS, THE MEASUREMENT THAT JUSTIFIES IT, AND WHAT THE ALTERNATIVE WOULD HAVE COST ON THE SAME HOST. COSTS ARE P99.9 UNLESS STATED.

Class	Adopted mechanism	Measured cost of the adopted mechanism	Cheapest rejected alternative, and why
A	Direct C ABI on the lane's CUDA stream; device pointers in, DU-owned buffers out; CUDA-event completion; grant shape checked against the declared admission profile	submit 19.9 μs P50, 27.7 μs P99.9 (estimator, live shape); receiver at the live shape 81.6 μs / 112 μs (P50 / P99.9)	Device-to-host export: 80.6 μs P99 at 1 MB before inference, plus a wake and a return copy no framework provides
B	Direct call at the decision boundary; intents range-checked and allow-flagged; deadline in the request, checked at commit; circuit breaker after 8 failures	0.288 μs over 20,000 validated calls (quiet), 5.22 μs with the DU on the air; 3.6 μs on the runtime	Same request over SCTP loopback: 15.6 μs quiet, 181 μs with the DU on the air; ZeroMQ `ipc://`: 21.3 μs and 787 μs
C native	Leased zero-copy views; latest-context cache with 3 bounded acquires	Idle stream: 1 atomic load, 0.048 μs P99.9; context acquire 48 ns P50, 0.27 μs P99.9	A copy per subscriber (the ring's 2.34 μs), paid only when isolation is wanted
C supervised	Shared ring: try-lock, reclaim read, 1 memcpy, release store, eventfd; full ring is a counted drop; SEQPACKET control with credentials; CUDA-IPC pool for device payloads	Producer at 68 KB: 2.34 μs, full-ring return 0.144 μs (full ring, counted drop); consumer wake 3.92 μs P50; control round trip 3.20 μs P50 (RTT/2)	ZeroMQ PUB/SUB at 68 KB: producer 3.14 μs P99.9; a slow subscriber's data ages to 981.95 ms; conflation drops silently
C portable	E3AP over SCTP (36423), APER via asn1c; FlatBuffers profile over SCTP (38472); credentialed local socket	Control round trip 10.9 μs P50 (RTT/2); 64 KB indication 255 μs one way; 1.47 MB grid 1.68 ms one way	None rejected: kept because 17 of 39 rows fit it and E2 tooling exists for it

A. Class A: A Stream, Not a Message

The inline rows have 50–300 μs budgets over 68 KB to several MB of GPU-resident tensors, and Section V-F shows that any path leaving the PHY's stream exceeds the budget before a single model FLOP runs. The only admissible mechanism is enqueue on the PHY's own stream: the ABI passes a `cudaStream_t` and device pointers, the DU records a completion event behind the dApp's work and keeps its conventional stage armed if the event has not fired by the deadline. Because the dApp is inside the process, the ABI is frozen and size-tagged, so a stale module is rejected at load, and the grant shape is checked against the module's declared admission profile before any module code runs.

B. Class B: A Call With a Contract

The bounded rows exchange kilobytes and wait for the answer, and Section V-D shows that every process boundary, however carried, moves the P99.9 from single microseconds to hundreds and adds misses at 100 μs once another thread shares the core. A direct call has no such tail, so the ABI uses one and spends the saved budget on what a boundary would have

provided: every intent is range-checked against the cell's PRB count, MCS table, and the operator's allow flags, the deadline travels in the request and is checked at commit, and a circuit breaker removes a module after 8 recoverable failures. That suffices because the outputs are 72 B each and fully validatable, and it is why the verified-hook alternative of Section V-E was evaluated and not adopted.

C. Class C: Three Positions, One Producer Rule

The nonblocking rows tolerate milliseconds but their producer does not: the thread publishing a spectrum grid is on the slot's critical path. Section V-C shows that producer cost and accountable staleness, not consumer latency, are the constraints, and each Class C position follows from them. *Leased views* cost the producer an atomic load when idle and a reference count when subscribed; the DU refuses to unload a module until its leases return, the quiescence proof of [8]. The *supervised process* is the shared ring, a try-lock, a reclaim read, 1 memcpy, a release store, and 1 eventfd write, with a full ring returning immediately as a counted drop, so staleness is bounded by depth and every loss is visible; control travels on a SEQPACKET socket with credentials verified per message, so a dead worker is detected within 1 heartbeat and restarted without touching the ring, and GPU payloads use a CUDA-IPC pool. *Portable E3* is the protocol-standard end: its measured costs (Section V-B) are inside every Class C budget, so it is a legitimate choice for all 17 rows and the only choice outside the host's trust domain, and the platform ships it with the same asn1c toolchain and message families as the Northeastern agent plus a FlatBuffers profile on a second association for bulk streams.

Supporting 3 positions costs the platform 3 egress implementations behind 1 publish call and the dApp author nothing: the package declares its position, and the price of looser coupling (Table III) is paid only by the dApp that chose it.

VII. Validation on the Released Platform

The mechanisms of Section VI ship in the public OCUDU dApp platform and SDK [18], [19]. Table IV places the runtime's own per-class checkpoints beside this study's measurement of the same mechanism on the same host, so the microbenchmark and the production number can be compared directly; the reference receiver's earlier 92.5 μs figure is reproduced at the live shape and, as Section V-F shows, does not yet extend to the 273-PRB kernels. The composition run in the last row is the practical test of the argument: 4 dApps from 3 classes, built out of tree against the installed SDK, loaded through 1 E3 agent onto 1 over-the-air n78 cell (GB10 host, USRP B210) with live UEs. The Class A equalizer replaced the conventional stage for every PUSCH grant, the Class B scheduler returned validated intents at every decision boundary, the Class C spectrum and SRS-ISAC dApps published at producer rate, one in-process and one as a supervised CUDA worker through the CUDA-IPC pool, and a Python E3 client subscribed to the same cell over SCTP from outside the process. The spectrum dApp's interference map reached the scheduler through the latest-context cache, the observation-to-control loop that no observer-only boundary closes inside the slot.

TABLE IV
Runtime checkpoints and the corresponding suite measurement. Runtime column: the platform's own benchmark binaries run on this host for this study, except the 92.5 / 105.1 μs, 45–52 μs, 3.6 μs, and 0.87 GB/s figures, which are from the release qualification. Suite column: this study, cell stopped, GPU exclusive.

Class	Mechanism	Runtime	Suite
A	Receiver completion at the live shape (51 PRB, 2 layers, 256-QAM), P50 / P99.9	92.5 / 105.1 μs	81.6 μs / 112 μs
A	Receiver completion at 273 PRB, 2 layers, P50 / P99.9 (150 μs deadline)	—	272 μs / 523 μs
A	Estimator submit on the lane's stream, live shape, P50 / P99.9	—	19.9 μs / 27.7 μs
A	Equalizer, live cell, P50	45–52 μs	—
B	Direct call P99.9 (runtime: worst profile, 9,000 calls; suite: 20,000 validated calls)	3.6 μs	0.288 μs
B	Latest-context acquire and release, P50 / P99.9	48 ns / 0.27 μs	—
C	Export across forced restarts; suite: ring producer per slot	0.87 Gb/s	2.05 μs / 3.89 μs
C	E3AP round trip: runtime get-config (local socket) P50 / P99; suite: control action over SCTP	13.0 μs / 51.7 μs	10.9 μs / 11.9 μs (RTT/2)
All	4 dApps, 3 classes, 1 OTA cell	260,000+ inv., 0 fallbacks	

VIII. Limitations

All measurements come from 1 host, an NVIDIA DGX Spark with a GB10 GPU and coherent CPU–GPU memory. On a PCIe-attached GPU the device-to-host rows move by the bus latency and the Class A argument only strengthens; the host-side rows should transfer with the usual caveat that tails depend on kernel configuration and isolation. Both host conditions are archived as raw traces.

The E3 rows measure the codec and the loopback carrier. A cross-host SCTP association adds network latency that depends entirely on the operator's fabric and is out of scope here. The 273-PRB/4-port rows are the same code at the envelope size and were not exercised over the air.

Class A is functionally resident and its interface cost is measured, but the reference receiver meets its deadline only at the live shape; the 273-PRB kernels are not yet qualified. Finally, the corpus verdicts are engineering judgments backed by byte and deadline arithmetic; a framework that adds an in-process position would move rows across the line, which is the point of the argument.

IX. Conclusion

The dApp tier was born as an observer with a reply channel, and for the sensing and advisory use cases that motivated it that boundary remains right. The audit here shows that more than half of the AI-RAN use cases now filed under the same label cannot cross it, for reasons of arithmetic: a copy and a wake on a slot's critical path, or a process hop inside a 100 μs decision, exceed the budget before the model runs. Measuring

the mechanisms the 3 public frameworks actually use prices each position on the coupling spectrum, and those prices derive the released ABI: a stream for Class A, a validated call for Class B, and for Class C a per-use-case choice among a lease, a supervised ring, and a protocol-standard E3 association. The released platform composes all 3 classes on 1 cell and records the checkpoints that let the argument be re-run.

Availability and Acknowledgment

The benchmark suite, corpus, raw CSVs, and this paper's sources accompany the released platform, SDK, and quickstart images, public under the OCUDU WG2 AI-RAN group [18]–[20]. This work is supported by the U.S. Department of Defense (DoD) Office of the Under Secretary of Defense for Research and Engineering (OUSD(R&E)) FutureG Office.

References

[1] O-RAN Alliance, "O-RAN architecture description," O-RAN Alliance, Tech. Rep. O-RAN.WG1.O-RAN-Architecture-Description-v11.00, 2024.

[2] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.

[3] S. D'Oro, M. Polese, L. Bonati, H. Cheng, and T. Melodia, "dApps: Distributed applications for real-time inference and control in O-RAN," *IEEE Communications Magazine*, 2022, arXiv:2203.02370.

[4] A. Lacava, L. Bonati, N. Mohamadi, R. Gangula, F. Kaltenberger, P. Johari, S. D'Oro, F. Cuomo, M. Polese, and T. Melodia, "dApps: Enabling real-time AI-based open RAN control," *Computer Networks*, vol. 269, p. 111342, 2025, arXiv:2501.16502.

[5] D. Villa, M. Belgiovine, N. Hedberg, M. Polese, C. Dick, and T. Melodia, "Programmable and GPU-accelerated edge inference for real-time ISAC on NVIDIA Aerial testbed," *arXiv preprint arXiv:2512.06493*, 2026.

[6] N. N. Santhi, D. Villa, M. Polese, and T. Melodia, "InterfO-RAN: Real-time in-band cellular uplink interference detection with GPU-accelerated dApps," *arXiv preprint arXiv:2507.23177*, 2025.

[7] M. Pennybacker, T. O'Shea, and A. Kharchenko, "Three classes of real-time AI-RAN dApps: Use-case and benchmark evidence for an expanded dApp definition," 2026, companion paper.

[8] ——, "An embedded dapp runtime for the OCUDU 5g nr du: Architecture for resident inline, bounded control, and asynchronous ai-ran applications," 2026, companion paper.

[9] WiNES Lab, "libe3: Vendor-neutral E3AP C++ library," GitHub repository, 2026. [Online]. Available: https://github.com/wineslab/libe3

[10] R. Gangula, A. Lacava, M. Polese, S. D'Oro, L. Bonati, F. Kaltenberger, P. Johari, and T. Melodia, "Listen-while-talking: Toward dApp-based real-time spectrum sharing in O-RAN," *arXiv preprint arXiv:2407.05027*, 2024.

[11] Microsoft, "jBPF: A userspace eBPF instrumentation framework for telecom applications," https://github.com/microsoft/jbpf, 2024.

[12] NVIDIA, "Aerial CUDA-accelerated RAN," https://docs.nvidia.com/aerial/, 2024.

[13] K. Cohen-Arazi, M. Roe, Z. Hu, R. Chavan, A. Ptasznik, J. Lin, J. Morais, J. Boccuzzi, and T. Balercia, "NVIDIA AI Aerial: AI-native wireless communications," *arXiv preprint arXiv:2510.01533*, 2025.

[14] R. Wiesmayr, C. Dick, J. Hoydis, and S. Cammerer, "Design of a standard-compliant real-time neural receiver for 5G NR," *arXiv preprint arXiv:2409.02912*, 2024.

[15] O-RAN Alliance, "O-RAN near-real-time RAN intelligent controller, E2 general aspects and principles (E2GAP)," O-RAN Alliance, Tech. Rep. O-RAN.WG3.E2GAP, 2024.

[16] R. Stewart, "Stream control transmission protocol," IETF, Tech. Rep. RFC 4960, 2007. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4960

[17] O-RAN Alliance nGRG, "dApps for real-time RAN control: Use cases and requirements," O-RAN Alliance, Tech. Rep. RR-2024-10, Oct. 2024.

[18] OCUDU WG2 AI-RAN, "ocudu-dapp-platform," https://gitlab.com/ocudu/work_groups/wg2_ai_ran/ocudu-dapp-platform, 2026.

[19] ——, "ocudu-dapp-sdk," https://gitlab.com/ocudu/work_groups/wg2_ai_ran/ocudu-dapp-sdk, 2026.

[20] ——, "ocudu-dapp-quickstart," https://gitlab.com/ocudu/work_groups/wg2_ai_ran/ocudu-dapp-quickstart, 2026.

APPENDIX
THE AUDITED USE-CASE CORPUS

TABLE V

RUNTIME ROWS OF THE AUDITED CORPUS (IDENTIFIERS ARE THE ONES USED IN THE TEXT). BYTES ARE PER INVOCATION AT THE LIVE 51-PRB, 2-PORT CELL AND AT THE 273-PRB, 4-PORT ENVELOPE (64 PORTS FOR A-04). VERDICTS FOR THE OBSERVER-ONLY BOUNDARY (E3 INDICATION OUT, CONTROL ACTION BACK) AND THE GPU-EXPORT BOUNDARY (COPY INTO A SECOND CONTEXT): *inexpr.*, NO RETURN PATH INTO THE CONSUMING STAGE; *infeas.*, A PATH EXISTS AND ITS MEASURED COST EXCEEDS THE BUDGET; *feas.*, FITS; *weak*, THE ROW DOES NOT DISCRIMINATE. STATUS IS ON THE RELEASED PLATFORM.

ID	Use case	Class	Live	Max	Deadline / freshness	Observer-only	GPU export	OCUDU status
A-01	Neural channel estimation	A	67 KB	1.4 MB	100 μs goal; 800 μs wall	inexpr.	inexpr.	shipped (0x00010003)
A-02	Neural equalization (CE included)	A	67 KB	1.4 MB	100 μs goal; 800 μs wall	inexpr.	inexpr.	45–52 μs P50 live; 260k+ inv.
A-03	Neural receiver to soft LLRs	A	67 KB	1.4 MB	150 μs goal; 2 ms wall	inexpr.	inexpr.	92.5 / 105.1 μs P50 / P99.9
A-04	Large-MIMO PUSCH at 64 rx ports	A	11.7 MB	22.4 MB	1 UL slot occupancy	inexpr.	inexpr.	proxy 453 μs P50, 1.4 ms P99
A-05	Joint neural EQ and FEC decoding	A	176 KB	4.4 MB	300 μs; soft buffer host-owned	inexpr.	inexpr.	out of scope (HARQ buffer)
A-06	Neural PRACH detection	A	13 KB	3.9 MB	1–2 ms per occasion	weak (rate)	weak	design only (no PRACH seam)
A-07	Neural SRS channel estimation	A	5 KB	210 KB	SRS period 5–40 ms	weak (size)	weak	design only as A; C path shipped
A-08	Inline neural precoding	A	68 KB	2.1 MB	DL slot; 100 μs goal	inexpr.	inexpr.	out of scope (no DL seam)
A-09	PAPR reduction and neural DPD	A	60 KB	240 KB	per symbol (35.7 μs)	inexpr.	inexpr.	RU-seam plugin path (vendor id)
A-10	ISAC waveform and beam writeback	A	68 KB	2.1 MB	DL slot	inexpr.	inexpr.	out of scope (sensing half is C-02)
A-11	Two-sided learned modem	A	68 KB	1.4 MB	50–300 μs per allocation	inexpr.	inexpr.	out of scope (UE pairing)
A-12	OTFS and Zak-OTFS air interface	A	68 KB	1.4 MB	100–500 μs per allocation	inexpr.	inexpr.	out of scope
A-13	Semantic joint source-channel coding	A	68 KB	1.4 MB	50–300 μs per allocation	inexpr.	inexpr.	out of scope (DL seam; paired UE model)
B-01	Scheduler intents	B	512 B	4 KB	100 μs to commit	infeas. (4×)	inexpr.	3.6 μs P99.9 runtime; 5.2 μs suite
B-02	Uplink power control (TPC)	B	64 B	64 B	100 μs (shared)	infeas.	inexpr.	shipped (TPC field)
B-03	Link adaptation and MCS	B	512 B	4 KB	100 μs (shared)	infeas.	inexpr.	shipped (OLLA reference)
B-04	Interference-aware PRB avoidance	B	275 B	275 B	100 μs to commit	infeas.	inexpr.	demonstrated live (K-01)
B-05	Per-slot slice enforcement	B	512 B	4 KB	100 μs (shared)	infeas.	inexpr.	shipped (fields + envelope)
B-06	Subband mute for SBFD / dynamic TDD	B	275 B	275 B	100 μs (shared)	infeas.	inexpr.	mask path shipped; host support open
B-07	Beam selection at the decision boundary	B	10 KB	10 KB	100 μs (shared)	infeas.	inexpr.	output defined; input context absent
B-08	Security-gated forbid	B	64 B	64 B	100 μs (shared)	infeas.	inexpr.	design only
B-09	Layer reduction for energy	B	512 B	4 KB	100 μs (layers)	infeas.	inexpr.	nof_layers intent; gain via C-16
C-01	Spectrum sensing and interference map	C	134 KB	1.4 MB	0.5–10 ms; per captured slot	feas. (decimated)	feas.	3 positions; 0.87 Gb/s across restarts
C-02	Quiet-period reservation / DSS windows	C	100 B	100 B	$\geq$ 1 slot ahead	feas. (no admission)	inexpr.	shipped (reservation authority)
C-03	SRS-ISAC feature extraction	C	5 KB	210 KB	$\leq$ SRS period	feas.	feas.	live (CUDA-IPC pool)
C-04	CSI prediction	C	5 KB	210 KB	1–10 ms; $\geq$ 5 ms over E3	feas.	feas.	data path shipped; consumer design
C-05	Advisory beam prediction	C	10 KB	10 KB	$\leq$ 1 slot native; 5–20 ms E3	feas.	feas.	design only
C-06	Beam failure prediction	C	1 KB	1 KB	10–200 ms timer	feas.	feas.	design only
C-07	Positioning and channel charting	C	5 KB	210 KB	10 ms–1 s	feas.	feas.	design only (srs.channel)
C-08	Positioning measurement enhancement	C	5 KB	210 KB	1–20 ms	feas.	feas.	design only
C-09	Joint sensing and positioning	C	5 KB	210 KB	10 ms–1 s	feas.	feas.	design only
C-10	RF fingerprinting	C	134 KB	2.9 MB	0.5–10 ms	feas.	feas.	design only (pusch.grid)
C-11	Physical-layer security detection	C	134 KB	1.4 MB	0.5–10 ms; gating via B-08	feas.	feas.	design only
C-12	Codebook and beam-weight optimisation	C	5 KB	210 KB	10 ms–1 s	feas.	feas.	design only (xApp candidate)
C-13	CSI feedback reconstruction	C	1 KB	1 KB	$\leq$ 1 slot fresh; 2 ms cadence	weak	weak	design only (no UCI seam)
C-14	Inter-cell interference coordination advisory	C	275 B	275 B	1–10 ms; multi-cell via E2	feas.	feas.	design only
C-15	RIS advisory	C	100 KB	100 KB	1–20 ms	feas.	feas.	out of scope
C-16	RF gain policy	C	1 KB	1 KB	1–10 ms actuation	feas.	feas.	actuation shipped (ranControl)
C-17	Portable external observer	C	134 KB	1.4 MB	stream floors	feas. (native)	feas.	live (Python E3 client)

Table V lists the 39 runtime rows and Table VI the management, xApp/rApp, and research rows that Section IV excludes from the runtime count; the identifiers are the ones used throughout the paper, and the mapping to the earlier corpus of [7] is kept in the repository. The byte counts are the inputs each row needs per invocation, computed from the 5G NR grid dimensions at 30 kHz subcarrier spacing; the deadline column gives the 5G NR bound or, for asynchronous rows, the freshness after which the result loses value. The verdict columns apply the arithmetic of Section V to the observer-only boundary (an E3 indication out and a control action back) and to a GPU-export boundary (device-to-host or device-to-device copy into a second context). The status column names what the released platform ships, has demonstrated on the air, defines but has not implemented, or leaves out of scope. The same rows, with their formulas and sources, are in the study repository's use-case inventory and performance matrix.

TABLE VI

ROWS THAT ARE NOT DAPPS: MANAGEMENT-PLANE APPLICATIONS ACTING THROUGH TYPED CONFIGURATION OR RAN CONTROL, XAPP/RAPP POLICY, AND 1 RESEARCH ROW THAT NEEDS A MULTI-DU SUBSTRATE. THEY CONSUME SUMMARIES OR POLICIES AND DO NOT SIZE THE MICROSECOND INTERFACES.

ID	Application	Bucket	Timescale	OCUDU status
X-01	QoS and slice-aware flow classification	xApp/rApp	1–10 ms	out of scope: no packet-path seam
X-02	RAN fault and anomaly detection	xApp/rApp	10 ms–1 s	live: MCP observe ring; ranControl actuation
X-03	Energy saving, carrier sleep, DTX/DRX	xApp/rApp	100 ms–1 s	partial: actuation shipped; policy out of scope
X-04	SON self-optimisation	xApp/rApp	> 1 s	out of scope: out of scope
X-05	Slice SLA management	xApp/rApp	> 1 s	out of scope: out of scope
X-06	Handover and load-balancing advisory	xApp/rApp	10 ms–1 s	partial: handoverUe / releaseUeToIdle shipped
X-07	Fronthaul and UL-throughput configuration	mgmt.	10 ms–1 s	partial: gain actuation shipped
X-08	Slice envelope	mgmt.	100 ms–1 s	shipped: typed configuration writes
X-09	LLM-agent operations (28 MCP tools)	mgmt.	seconds	live: MCP server on the E3AP endpoint
R-02	Distributed and cell-free MIMO	research	per slot / 100 ms	out of scope: multi-DU substrate